SCORES: Shelf Trajectories with Slow and Fast Components via Spectral Quadratic Surrogates

Monysovann Ly

February 19, 2026

Abstract

We study online reordering of a finite set of items to reduce user navigation time while remaining consistent with a drifting physical shelf layout. We argue that raw swipe adjacency is a policy artifact and propose learning proximity only from consecutive *update events*. To obtain a solver with clean spectral structure, we replace absolute distance costs by a Laplacian quadratic surrogate that diagonalizes in the graph Fourier basis [1, 7]. We then formulate a two time scale trajectory problem, with a slow shelf component and a fast app component coupled by a proximity term and controlled by temporal smoothness. The resulting regularizers diagonalize in the time frequency domain, yielding an explicit low pass interpretation for shelf drift [5, 6]. A practical solution is a two stage pipeline: spectral embeddings for daily structure, temporal smoothing for shelf inference, then constrained local swaps to produce valid permutations with disruption caps and protection for cold items.

Keywords. Online reordering; spectral seriation; Laplacian quadratic surrogate; time frequency smoothing; drifting shelf inference; constrained swaps.

Contents

1 Setup **2**
- 1.1 Items and permutations . . . 2
- 1.2 Sessions, updates, and why swipe adjacency is not the right signal . . . 2
- 1.3 Update adjacency weights . . . 2
- 1.4 Two half lives: fast evidence and slow evidence . . . 3

2 Quadratic surrogate that diagonalizes on graphs **3**
- 2.1 Laplacians . . . 3
- 2.2 From embeddings to permutations . . . 3

3 Trajectory formulation with slow shelf and fast app components **3**
- 3.1 State variables . . . 3
- 3.2 Time domain objective . . . 4

4 Frequency domain view of shelf drift **4**

5 Practical solver **5**
5.1 Stage 1: build Laplacians from update events 5
5.2 Stage 2: spectral embeddings 5
5.3 Stage 3: infer a drifting shelf by time frequency smoothing 5
5.4 Stage 4: inject fast improvements onto the shelf with hard caps 5
5.5 Complexity 6

6 Notes on calibration **6**
6.1 Hyperparameters to calibrate 6
6.2 Online hyperparameter calibration via a multi-armed bandit 6
6.3 Half lives 8
6.4 Cold item weights 8

7 Conclusion **8**

1 Setup

1.1 Items and permutations

Let $[n] := \{1, \dots, n\}$ index items. An ordering is a permutation π of $[n]$. Write $\mathrm{pos}_\pi(i) \in [n]$ for the position of item i under π.

1.2 Sessions, updates, and why swipe adjacency is not the right signal

A session is a sequence of visited items $(v_1, \dots, v_m)$ with action indicators $a_t \in \{0, 1\}$ where $a_t = 1$ denotes an *update event*. The swipe path (v_t) depends strongly on the current UI order, so transition counts between consecutive swipes largely reflect current adjacency rather than user intent.

We therefore base learning on the subsequence of *consecutive updates*. Let $t_1 < \cdots < t_k$ be indices with $a_{t_r} = 1$. Define the update sequence

$$u_r := v_{t_r}, \qquad r = 1, \dots, k.$$

Optionally, collapse consecutive repeats by removing u_{r+1} whenever $u_{r+1} = u_r$. Let $\Delta_r \geq 0$ be the time between update events r and $r + 1$.

1.3 Update adjacency weights

Define a robust weight function $w(\Delta)$, for example $w(\Delta) = 1$ for counts, or $w(\Delta) = \min(\Delta, c)$ for clipped time. For a day index d, define symmetric update adjacency weights

$$\widehat{S}_{ij}(d) := \sum_{\text{sessions on day } d} \sum_{r=1}^{k-1} \mathbf{1}\{u_r = i,\ u_{r+1} = j\}\, w(\Delta_r) \ + \ \mathbf{1}\{u_r = j,\ u_{r+1} = i\}\, w(\Delta_r).$$

This captures proximity only between items that are edited consecutively, which is the quantity the UI order can realistically reduce.

1.4 Two half lives: fast evidence and slow evidence

To support two time scales, maintain two exponentially weighted estimates:

$$S^{\text{fast}}(d) = \rho_f S^{\text{fast}}(d-1) + (1-\rho_f)\widehat{S}(d), \qquad S^{\text{slow}}(d) = \rho_s S^{\text{slow}}(d-1) + (1-\rho_s)\widehat{S}(d),$$

with $0 < \rho_f < \rho_s < 1$. ρ_f reacts quickly to short term changes and ρ_s captures persistent structure.

For cold item protection we also maintain a decayed update mass

$$c_i(d) = \rho_c c_i(d-1) + (1-\rho_c)\widehat{c}_i(d), \qquad \widehat{c}_i(d) := \sum_{\text{sessions on day } d} \sum_{r=1}^{k} \mathbf{1}\{u_r = i\}.$$

2 Quadratic surrogate that diagonalizes on graphs

2.1 Laplacians

Given a symmetric similarity matrix $S(d)$, define degree $D(d) = \text{diag}(S(d)\mathbf{1})$ and Laplacian

$$L(d) := D(d) - S(d).$$

For an embedding vector $x \in \mathbb{R}^n$,

$$x^\top L(d) x = \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} S_{ij}(d)\,(x_i - x_j)^2.$$

This is the squared distance surrogate. It is quadratic and diagonalizes in the eigenbasis of $L(d)$. Such Laplacian quadratic forms underpin many spectral embedding and clustering relaxations [1, 7, 4].

2.2 From embeddings to permutations

Given an embedding x, define the induced ordering

$$\pi(x) := \text{argsort}(x),$$

sorting items by x_i. Spectral seriation uses the Fiedler vector, an eigenvector associated with the second smallest eigenvalue of $L(d)$:

$$x^\star(d) \in \arg \min_{x \perp \mathbf{1},\ \|x\|_2 = 1} x^\top L(d) x.$$

We use $L^{\text{fast}}(d)$ for daily structure and $L^{\text{slow}}(d)$ for shelf structure. For additional seriation-oriented discussion and tooling, see Concas et al. [2, 3].

3 Trajectory formulation with slow shelf and fast app components

3.1 State variables

For each day $d \in \{0, \ldots, D-1\}$:

1. $x_a(d) \in \mathbb{R}^n$ is the fast app embedding.
2. $x_s(d) \in \mathbb{R}^n$ is the slow shelf embedding.
3. $\pi_a(d) = \pi(x_a(d))$ is the shown order after rounding and constraints.
4. $\pi_s(d) = \pi(x_s(d))$ is the inferred shelf order after rounding.

3.2 Time domain objective

We propose the following quadratic trajectory objective:

$$\min_{\{x_a(d),x_s(d)\}} \sum_{d=0}^{D-1} x_a(d)^\top L^{\text{fast}}(d)\, x_a(d) \;+\; \sum_{d=0}^{D-1} x_s(d)^\top L^{\text{slow}}(d)\, x_s(d) \tag{1}$$

$$+ \sum_{d=0}^{D-1} \|x_a(d) - x_s(d)\|^2_{\Gamma(d)} \tag{2}$$

$$+ \sum_{d=1}^{D-1} \|x_s(d) - x_s(d-1)\|^2_{K(d)} \;+\; \sum_{d=2}^{D-1} \|x_s(d) - 2x_s(d-1) + x_s(d-2)\|^2_{H(d)}. \tag{3}$$

Constraints per day:

$$x_a(d) \perp \mathbf{1},\ \|x_a(d)\|_2 = 1, \qquad x_s(d) \perp \mathbf{1},\ \|x_s(d)\|_2 = 1.$$

Here $\|z\|^2_M := z^\top M z$ with diagonal positive semidefinite matrices:

1. $\Gamma(d)$ couples app and shelf.
2. $K(d)$ penalizes shelf velocity.
3. $H(d)$ penalizes shelf acceleration.

Item wise cold protection enters by setting diagonal weights as increasing functions of low activity. A simple choice is

$$\Gamma_{ii}(d) = \gamma\, \lambda_i(d), \quad K_{ii}(d) = \kappa\, \lambda_i(d), \quad H_{ii}(d) = \eta\, \lambda_i(d),$$

with $\lambda_i(d)$ large when $c_i(d)$ is small, for example

$$\lambda_i(d) = \lambda_{\max} \exp(-\beta c_i(d)).$$

Cold items then resist both injection and drift.

4 Frequency domain view of shelf drift

This section explains the slow and fast split in time frequency terms. For clarity, ignore the graph data terms (1) and consider the shelf smoothing subproblem with $x_a(d)$ fixed:

$$\min_{\{x_s(d)\}} \sum_{d=0}^{D-1} \|x_a(d) - x_s(d)\|^2_\Gamma + \sum_{d=1}^{D-1} \|x_s(d) - x_s(d-1)\|^2_K + \sum_{d=2}^{D-1} \|x_s(d) - 2x_s(d-1) + x_s(d-2)\|^2_H, \tag{4}$$

with constant diagonal weights Γ, K, H for exposition.

Let $X_a(\omega)$ and $X_s(\omega)$ be the discrete Fourier transforms over d applied coordinate wise. Define

$$g(\omega) = 4 \sin^2(\pi\omega/D).$$

Then the first difference term contributes a factor $g(\omega)$ and the second difference contributes $g(\omega)^2$. This "trend–cycle" style smoothing viewpoint is closely related to second-difference penalization as in the Hodrick–Prescott filter [5, 6]. The objective (4) diagonalizes by ω and by coordinate:

$$\sum_{\omega=0}^{D-1} \left(\|X_a(\omega) - X_s(\omega)\|^2_\Gamma + g(\omega) \|X_s(\omega)\|^2_K + g(\omega)^2 \|X_s(\omega)\|^2_H \right).$$

The minimizer has the explicit low pass form, coordinate wise:

$$X_{s,i}(\omega) = \frac{\Gamma_{ii}}{\Gamma_{ii} + K_{ii}\, g(\omega) + H_{ii}\, g(\omega)^2}\; X_{a,i}(\omega).$$

Low frequencies with small $g(\omega)$ pass into the shelf, high frequencies are suppressed. The residual $X_a(\omega) - X_s(\omega)$ is a complementary high pass, which is the fast component injected into the UI.

5 Practical solver

5.1 Stage 1: build Laplacians from update events

On each day d:

1. Parse sessions to obtain update sequences (u_r) and inter update times Δ_r.
2. Accumulate $\widehat{S}(d)$ via consecutive updates.
3. Update $S^{\text{fast}}(d)$ and $S^{\text{slow}}(d)$ with two forgetting factors.
4. Form Laplacians $L^{\text{fast}}(d)$ and $L^{\text{slow}}(d)$.

5.2 Stage 2: spectral embeddings

Compute daily fast embedding $x_a^{(0)}(d)$ as the Fiedler vector of $L^{\text{fast}}(d)$. Compute daily slow embedding $x_s^{(0)}(d)$ as the Fiedler vector of $L^{\text{slow}}(d)$.

These are graph frequency solutions, diagonalized by the eigenbasis of each Laplacian.

5.3 Stage 3: infer a drifting shelf by time frequency smoothing

Use (4) to smooth the slow embedding trajectory over time. There are two equivalent implementations:

1. Frequency domain: take the DFT over d, apply the low pass gain above coordinate wise, then invert the DFT.
2. Time domain: solve the banded normal equations induced by (4), separately for each coordinate.

Denote the smoothed result by $x_s(d)$ and set $\pi_s(d) = \pi(x_s(d))$.

5.4 Stage 4: inject fast improvements onto the shelf with hard caps

We produce a valid shown permutation by starting from the shelf order and applying a bounded number of local moves guided by the fast objective. Let $\pi \leftarrow \pi_s(d)$.

Define a discrete fast score for swaps using the update adjacency weights:

$$\widetilde{J}_{\text{fast}}(\pi; d) := \sum_{i<j} S_{ij}^{\text{fast}}(d)\; \big(\text{pos}_\pi(i) - \text{pos}_\pi(j)\big)^2,$$

which is consistent with the quadratic surrogate.

Apply constrained local search:

1. Choose a daily swap budget B and a window radius w.

2. Restrict candidate swaps to pairs within w positions in the current order.
3. Enforce a per item displacement cap $r_i(d)$ relative to $\pi_s(d)$, with small caps for cold items.
4. Accept a swap only if it reduces $\widetilde{J}_{\text{fast}}(\pi; d)$ and respects caps.

Return $\pi_a(d) = \pi$.

This injection step yields fast adaptation without rewriting the shelf. Cold items remain stable because they rarely participate in beneficial swaps and their displacement caps are small.

5.5 Complexity

For sparse $S(d)$ the Laplacian is sparse. Computing a Fiedler vector can be done with iterative methods whose dominant cost is sparse matrix vector products. The swap phase can be implemented with incremental score updates that touch only edges incident to swapped items.

6 Notes on calibration

6.1 Hyperparameters to calibrate

This model has several tunable hyperparameters that control (i) how evidence is aggregated in time, (ii) how strongly the shelf is smoothed, and (iii) how disruptive the fast "injection" step is allowed to be. The main quantities to calibrate are:

- **Update-pair weighting:** parameters inside $w(\Delta)$ (e.g., clip level c if using $w(\Delta) = \min(\Delta, c)$, or any other robust time-to-weight map).
- **Fast/slow evidence time scales:** forgetting factors ρ_f (fast) and ρ_s (slow).
- **Cold-item mass time scale:** forgetting factor ρ_c used in $c_i(d)$.
- **Shelf smoothing strength:** the trajectory regularization weights in the shelf smoother (e.g., λ_1, λ_2 in (4), or equivalently any frequency-domain low-pass cutoff parameter).
- **Swap-search aggressiveness:** daily swap budget B and window radius w.
- **Displacement-cap schedule:** cap endpoints $r_{\min}, r_{\max}$ and the coldness scale τ_c used in $r_i(d)$.

6.2 Online hyperparameter calibration via a multi-armed bandit

We calibrate the SCORES hyperparameters online using a multi-armed bandit (MAB). Each arm corresponds to a full configuration of discrete hyperparameter choices drawn from plausible ranges. The bandit chooses a configuration each day, we run the SCORES solver with that configuration, deploy the resulting ordering, then observe a binary improvement label from user interaction metrics.

Arms as discrete configurations. Let θ denote the vector of tunable hyperparameters, for example clipping inside $w(\Delta)$, forgetting factors (ρ_f, ρ_s, ρ_c), shelf smoothing strengths, swap budget B, window radius w, and displacement cap parameters. For each coordinate θ_j, choose a small discrete set $\mathcal{G}_j$ of plausible values. The candidate configuration set is then

$$\Theta := \mathcal{G}_1 \times \cdots \times \mathcal{G}_p.$$

Each arm $a \in [A]$ corresponds to one configuration $\theta^{(a)} \in \Theta$. If $|\Theta|$ is too large, we subsample a manageable set of arms by randomized selection or a space filling design over the discrete grid.

Daily decision loop. On day d, the bandit selects an arm a_d and therefore a configuration $\theta^{(a_d)}$. We then run the full SCORES pipeline using $\theta^{(a_d)}$ and deploy the resulting shown permutation:

$$\pi_a(d) \;=\; \mathrm{SCORES}\big(\widehat{S}(d),\; S^{\mathrm{fast}}(d),\; S^{\mathrm{slow}}(d),\; c(d);\; \theta^{(a_d)}\big).$$

All safety constraints remain enforced by the SCORES solver, including bounded swap budgets and per item displacement caps.

Binary improvement label as reward. Let $T(d)$ be the total time spent in the update view on day d, and let $W(d)$ be the total number of swipes on day d. We define a binary ground truth improvement label

$$y_d := \mathbf{1}\Big\{T(d) < T(d-1) \;\;\text{or}\;\; W(d) < W(d-1)\Big\},$$

with $y_d = 0$ otherwise. This label serves as the bandit reward. The goal is to choose configurations that maximize the probability of improvement.

Bandit update and selection rule. We model each arm as a Bernoulli process with unknown success probability p_a. A simple and effective choice is Thompson sampling with independent Beta posteriors:

$$p_a \sim \mathrm{Beta}(\alpha_a, \beta_a).$$

Initialize (α_a, β_a) to small pseudo counts, for example $\alpha_a = \beta_a = 1$. Each day:

1. For each arm a, sample $\widetilde{p}_a \sim \mathrm{Beta}(\alpha_a, \beta_a)$.
2. Select $a_d \in \arg\max_a \widetilde{p}_a$.
3. Deploy SCORES using $\theta^{(a_d)}$ and observe y_d.
4. Update only the played arm:

$$\alpha_{a_d} \leftarrow \alpha_{a_d} + y_d, \qquad \beta_{a_d} \leftarrow \beta_{a_d} + (1 - y_d).$$

This produces an exploration exploitation tradeoff automatically, favoring arms with high estimated improvement probability while still exploring uncertain arms.

Stopping and locking the configuration. After an exploration period of D_{explore} days, we choose a single configuration for stable deployment, for example

$$\widehat{a} \in \arg\max_a \mathbb{E}[p_a] = \arg\max_a \frac{\alpha_a}{\alpha_a + \beta_a},$$

and then run SCORES with $\theta^{(\widehat{a})}$ as the default. Optionally, we continue to run the bandit at a low exploration rate to adapt to long run drift in user behavior.

Remark on nonstationarity. Because user behavior and inventory regimes can drift, a discounted or sliding window update can be used to emphasize recent outcomes. One simple approach is to apply exponential forgetting to the pseudo counts before adding the new observation, which yields a recency weighted bandit that tracks gradual change.

6.3 Half lives

Set ρ_f with a short half life that matches behavioral drift, such as days. Set ρ_s with a long half life that matches shelf drift, such as weeks or months.

6.4 Cold item weights

Use $c_i(d)$ to define $\lambda_i(d)$, then derive:

$$r_i(d) = r_{\min} + (r_{\max} - r_{\min}) \frac{c_i(d)}{c_i(d) + \tau_c},$$

so hot items may move more and cold items stay near shelf.

7 Conclusion

SCORES is formulated as a two time scale trajectory problem. Update to update adjacency provides an intent aligned signal and avoids self reinforcing swipe adjacency. A Laplacian quadratic surrogate yields spectral diagonalization over items, and temporal difference penalties yield diagonalization over time frequencies. The resulting system supports a drifting shelf estimate and bounded fast injections into the shown ordering with explicit stability controls.

References

[1] Mikhail Belkin and Partha Niyogi. Laplacian eigenmaps and spectral techniques for embedding and clustering. In *Advances in Neural Information Processing Systems 14*, 2002. NIPS 2001.

[2] Anna Concas, Caterina Fenu, and Giuseppe Rodriguez. PQSER: A Matlab package for spectral seriation. *arXiv preprint arXiv:1711.05677*, 2017.

[3] Anna Concas, Caterina Fenu, Giuseppe Rodriguez, and Raf Vandebril. The seriation problem in the presence of a double Fiedler value. *arXiv preprint arXiv:2204.03362*, 2022.

[4] Inderjit S. Dhillon, Yuqiang Guan, and Brian Kulis. Kernel k-means, spectral clustering and normalized cuts. *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2004. Often cited for the equivalence between weighted kernel k-means and normalized cuts relaxations.

[5] Robert J. Hodrick and Edward C. Prescott. Postwar u.s. business cycles: An empirical investigation. *Journal of Money, Credit and Banking*, 1997.

[6] Morten O. Ravn and Harald Uhlig. On adjusting the Hodrick–Prescott filter for the frequency of observations. *The Review of Economics and Statistics*, 2002.

[7] Jianbo Shi and Jitendra Malik. Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2000.